AURORA SIGER

Módulo de Gerenciamento de Pouso e Estabilização de Base (MGPEB)

A Aurora Ajusta a Trajetória na Aproximação a Marte

Relatório técnico do sistema embarcado de pouso

Descrição do cenário, regras de decisão com portas lógicas, estruturas de dados, algoritmos de busca e ordenação, modelagem matemática, contexto histórico e reflexão ESG, com a explicação do código Python do protótipo (`mgpeb.py e mgpeb.ipynb`).

Curso	Ciência da Computação – FIAP
Atividade	Atividade integradora da Fase 2 (CAP 1) – PBL Aurora Siger
Disciplinas integradas	Computer Science (portas lógicas), Pensamento Computacional e Automação com Python, Data Structures and Algorithms, Modelagem Matemática e Computacional, Computer Organization and Architecture, Formação Social e Sustentabilidade
Integrantes da equipe	Henrique Floriano Alcantara - RM 575651 Katia Regina Bispo - RM 576044 João Pedro Candido Souza - RM 575884 Nicolas Klai de de França - RM 575343 Yago Souza Araujo - RM 575200
Data	Outubro de 2026

Entregáveis relacionados: este relatório (PDF), o código-fonte (`mgpeb.py e mgpeb.ipynb`) e o anexo de estruturas de dados (Anexo A, ao final deste documento).

GITHUB

https://github.com/Henrique-Alcantara07/MGPEB

1. Introdução

Pousar em outro planeta é muito mais do que "encostar no solo". Em missões como a do rover Curiosity, em 2012, a entrada, descida e pouso ocorreu como uma sequência automática de eventos, sem intervenção humana direta, por causa do atraso de comunicação entre Marte e a Terra. Por trás disso existe lógica booleana, estruturas de dados, algoritmos eficientes, modelos matemáticos e hardware robusto.

Um exemplo histórico inspirou o desenho do nosso sistema. Na descida do módulo lunar da Apollo 11, em 1969, o computador de bordo emitiu alarmes de sobrecarga (códigos 1201 e 1202). O software de bordo, cuja engenharia era liderada por **Margaret Hamilton** no MIT, havia sido projetado para descartar tarefas de menor prioridade e manter as críticas em execução, e o pouso prosseguiu com segurança. A lição é que, em sistemas críticos, **decidir o que vem primeiro** e **quais condições bloqueiam uma ação** é tão importante quanto o cálculo em si.

O **MGPEB** (Módulo de Gerenciamento de Pouso e Estabilização de Base) aplica essa ideia à missão Aurora Siger: organiza a ordem de pouso dos módulos da colônia por prioridade, verifica condições de segurança com lógica booleana, registra cada decisão em um histórico e modela o consumo de combustível. O módulo não controla foguetes reais. É um protótipo didático, mas tecnicamente coerente.

2. Cenário de pouso e módulos definidos

Depois de meses de viagem, a nave Aurora Siger entra na órbita de Marte. Cinco módulos automatizados serão desacoplados da nave-mãe e enviados à região escolhida para a colônia. Cada módulo é descrito por nove atributos, três deles booleanos (condições de segurança simuladas).

Módulo	Prior.	Comb. (%)	Massa (t)	Critic.	Chegada	Atmosf. OK	Área livre	Sensores OK
Habitação	4	85	20	4	10:00	Sim	Sim	Sim
Energia	5	92	15	5	10:15	Sim	Sim	Sim
Suporte à Vida	5	80	10	5	10:30	Sim	**Não**	Sim
Científico	2	78	12	2	10:45	Sim	Sim	Sim
Logística	3	90	18	3	11:00	Sim	Sim	**Não**

Tabela 1 – Módulos cadastrados no protótipo (prioridade e criticidade de 1 a 5; 5 é o mais crítico).

Energia e Suporte à Vida recebem prioridade 5 porque sem eles os demais módulos não funcionam e a tripulação não sobrevive. Habitação vem em seguida, e Logística e Científico ficam por último. Os valores *Suporte à Vida sem área livre*, *Logística com sensores comprometidos* e *Científico com 78% de combustível* foram escolhidos de propósito para exercitar cada alerta do sistema.

3. Visão geral do funcionamento

O programa executa sete etapas em sequência. A tabela relaciona cada ação com a estrutura de dados ou o algoritmo responsável.

Etapa	Ação do sistema	Função no código	Conceito aplicado
1	Exibir os módulos cadastrados	exibir_modulos()	Lista de listas
2	Ordenar a fila de pouso por prioridade (maior primeiro)	selection_sort()	Ordenação por seleção
3	Ordenar cópia da lista por nome (A a Z)	insertion_sort ()	Ordenação por inserção
4	Processar a fila, verificar as 4 condições e registrar eventos	processar_pousos()	Fila FIFO, pilha LIFO, AND/NOT
5	Mostrar resumo e histórico de eventos	exibir_resumo(), exibir_pilha()	Listas auxiliares, pilha
6	Buscar um módulo pelo nome	busca_linear(), busca_binaria()	Busca sequencial e binária
7	Plotar o consumo de combustível	exibir_grafico_combustivel()	Função linear (NumPy/Matplotlib)

Tabela 2 – Fluxo de execução do MGPEB.

Resultado com os dados de exemplo. Dos cinco módulos, dois pousam e três ficam em alerta:

Ordem	Módulo	Resultado	Motivo
1	Energia	Pouso autorizado	Todas as condições atendidas
2	Suporte à Vida	Alerta	Área de pouso indisponível
3	Habitação	Pouso autorizado	Todas as condições atendidas
4	Logística	Alerta	Sensores comprometidos
5	Científico	Alerta	Combustível de 78% (abaixo de 80%)

Tabela 3 – Resultado da simulação, na ordem em que a fila foi atendida.

4. Regras de decisão com portas lógicas

Cada decisão de pouso depende de quatro variáveis booleanas. Três já vêm do cadastro do módulo e uma é calculada por comparação relacional:

Variável	Origem	Significado
combustivel_ok	modulo[2] >= 80	Verdadeiro se o combustível é de pelo menos 80%
atmosfera_ok	modulo[6]	Condições atmosféricas simuladas adequadas
area_disponivel	modulo[7]	Área de pouso livre
sensores_ok	modulo[8]	Integridade dos sensores

A **função booleana** de autorização é **AUTORIZADO = C · A · L · S** (E lógico das quatro variáveis, com C = combustivel_ok, A = atmosfera_ok, L = area_disponivel e S = sensores_ok). Para cada condição que falha, a porta **NOT** gera o alerta correspondente: $ALERTA_x$ = NOT x. O circuito completo está na Figura 1.

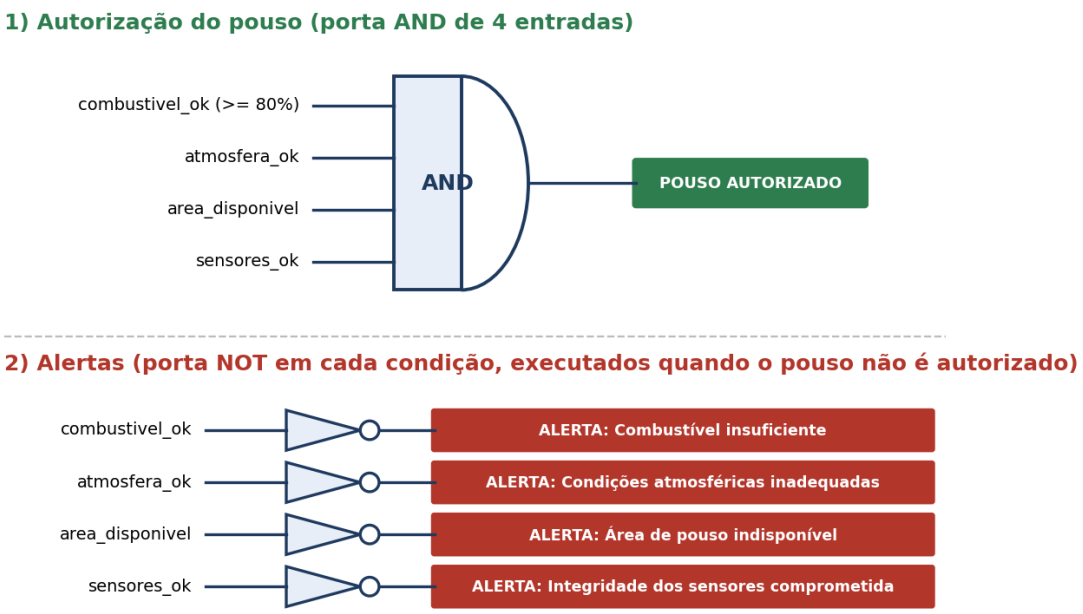


Figura 1 – Circuito lógico do processamento de pouso: AND para autorizar, NOT para os alertas. Fonte: elaborada pela equipe (2026).

Tabela-verdade da função (16 combinações). Apenas a combinação 1111 autoriza o pouso:

#	C	A	L	S	Resultado	Alertas gerados
0	0	0	0	0	Não autorizado	combustível, atmosfera, área, sensores
1	0	0	0	1	Não autorizado	combustível, atmosfera, área
2	0	0	1	0	Não autorizado	combustível, atmosfera, sensores
3	0	0	1	1	Não autorizado	combustível, atmosfera
4	0	1	0	0	Não autorizado	combustível, área, sensores
5	0	1	0	1	Não autorizado	combustível, área
6	0	1	1	0	Não autorizado	combustível, sensores
7	0	1	1	1	Não autorizado	combustível
8	1	0	0	0	Não autorizado	atmosfera, área, sensores
9	1	0	0	1	Não autorizado	atmosfera, área
10	1	0	1	0	Não autorizado	atmosfera, sensores
11	1	0	1	1	Não autorizado	atmosfera
12	1	1	0	0	Não autorizado	área, sensores
13	1	1	0	1	Não autorizado	área
14	1	1	1	0	Não autorizado	sensores
15	1	1	1	1	Autorizado	nenhum

Tabela 4 – Tabela-verdade da regra de autorização.

Tradução para o código. A porta AND vira o operador `and` e cada porta NOT vira `if not`. O bloco abaixo (função `processar_pousos`) mostra as duas partes:

```
combustivel_ok  = modulo[2] >= 80      # comparação relacional
atmosfera_ok    = modulo[6]
area_disponivel = modulo[7]
sensores_ok     = modulo[8]

if combustivel_ok and atmosfera_ok and area_disponivel and sensores_ok:   # AND
    pousados.append(modulo)
    pilha.append(f"{modulo[0]}: pouso autorizado")
else:
    if not combustivel_ok:  print(">> ALERTA: Combustível insuficiente")  # NOT
    if not atmosfera_ok:    print(">> ALERTA: Condições atmosféricas inadequadas")
    if not area_disponivel: print(">> ALERTA: Área de pouso indisponível")
    if not sensores_ok:     print(">> ALERTA: Integridade dos sensores comprometida")
    alertas.append(modulo)
    pilha.append(f"{modulo[0]}: pouso não autorizado")
```

Note que o operador `>=` torna o limite de 80% *inclusivo*: o módulo Suporte à Vida, com exatamente 80%, passa na condição de combustível. Os alertas são independentes, então um mesmo módulo pode acumular vários.

5. Estruturas de dados lineares

O sistema usa listas simples do Python em três papéis diferentes. Os detalhes e exemplos concretos estão no Anexo A; aqui está o resumo:

Estrutura	Variável	Regra de acesso	Para que serve
Lista de listas	modulos	Acesso por índice	Cadastro original: cada módulo é uma lista de 9 campos
Fila (FIFO)	fila_pouso	append() entra no fim, pop(0) sai do início	Ordem de atendimento dos pousos
Pilha (LIFO)	pilha_eventos	append() empilha, reversed() lê do topo	Histórico: evento mais recente aparece primeiro
Listas auxiliares	modulos_pousados, modulos_alerta, modulos_alfabeticos	append()	Resultados e cópia ordenada por nome para a busca binária

Tabela 5 – Estruturas de dados do protótipo.

6. Algoritmos de ordenação e busca

6.1 Selection Sort: ordenar a fila por prioridade

A cada passada, o algoritmo procura o maior valor de prioridade (índice 1) no trecho ainda desordenado e o troca para a posição atual. Resulta em ordem decrescente de prioridade.

```
def selection_sort(fila):
    n = len(fila)
    for i in range(n):
        maior = i
        for j in range(i + 1, n):
            if fila[j][1] > fila[maior][1]:   # compara prioridades
                maior = j
        fila[i], fila[maior] = fila[maior], fila[i]   # troca de posição
```

Passada (i)	Maior encontrado	Estado da fila após a passada
início	-	Habitação, Energia, Suporte à Vida, Científico, Logística
0	Energia (5)	**Energia**, Habitação, Suporte à Vida, Científico, Logística
1	Suporte à Vida (5)	Energia, **Suporte à Vida**, Habitação, Científico, Logística
2	Habitação (4)	Sem troca (já está na posição certa)
3	Logística (3)	Energia, Suporte à Vida, Habitação, **Logística**, Científico

Tabela 6 – Execução passo a passo do Selection Sort com os dados do projeto. Em caso de empate (Energia e Suporte à Vida, ambos 5), o algoritmo mantém a que já estava antes, pois o teste usa > e não >=.

6.2 Insertion Sort: ordenar por nome

Pega cada elemento e o insere na posição correta dentro da parte já ordenada, deslocando os maiores para a direita. A comparação usa `casefold()`, que ignora maiúsculas e minúsculas. Resultado: Científico, Energia, Habitação, Logística, Suporte à Vida. Essa ordem alfabética é **pré-requisito da busca binária**.

```
def insertion_sort(lista):
    for i in range(1, len(lista)):
        atual = lista[i]
        j = i - 1
        while j >= 0 and lista[j][0].casefold() > atual[0].casefold():
            lista[j + 1] = lista[j]     # desloca para a direita
            j -= 1
        lista[j + 1] = atual            # insere na posição correta
```

6.3 Busca linear e busca binária

A **busca linear** confere os módulos um por um e funciona em qualquer lista. A **busca binária** olha o elemento do meio, descarta a metade onde o nome não pode estar e repete; exige lista ordenada (por isso o Insertion Sort). Procurando "energia" na lista alfabética: meio = Habitação (energia vem antes, descarta a direita), meio

= Científico (energia vem depois, descarta a esquerda), meio = Energia (encontrado), em 3 passos. Com apenas 5 módulos a diferença para a busca linear é pequena, mas ela cresce rapidamente com listas maiores.

```
def busca_binaria(modulos, nome):
    inicio, fim = 0, len(modulos) - 1
    nome_busca = nome.casefold()
    while inicio <= fim:
        meio = (inicio + fim) // 2
        nome_meio = modulos[meio][0].casefold()
        if nome_meio == nome_busca:
            return modulos[meio]
        elif nome_busca > nome_meio:
            inicio = meio + 1          # descarta a metade esquerda
        else:
            fim = meio - 1             # descarta a metade direita
    return None
```

Algoritmo	Complexidade	Justificativa
Selection Sort	O(n²)	Laço dentro de laço: n(n-1)/2 comparações, sempre (10 para n = 5)
Insertion Sort	O(n²) no pior caso	Pior caso: lista invertida. Se já estiver ordenada, é O(n)
Busca linear	O(n)	Pior caso: percorre todos os elementos
Busca binária	O(log n)	Divide a lista ao meio a cada passo

Tabela 7 – Complexidade dos algoritmos (n = número de módulos).

Para uma frota de cinco módulos, os algoritmos O(n²) são perfeitamente adequados, e sua simplicidade os torna fáceis de testar e verificar, qualidade valiosa em sistemas críticos.

7. Modelagem matemática do combustível

Escolhemos o fenômeno **consumo de combustível durante a descida**, representado por uma **função linear decrescente**:

$$C(t) = C_0 - a \cdot t$$

Parâmetro	Valor	Significado
C(t)	variável	Combustível restante (%) no instante t
C_0	90%	Combustível no início da descida (coeficiente linear: onde a reta corta o eixo vertical)
a	2% por minuto	Taxa de consumo (coeficiente angular, com sinal negativo: a reta é decrescente)
t	0 a 30 min	Tempo de descida
C_{min}	80%	Limite mínimo exigido pela regra lógica de autorização

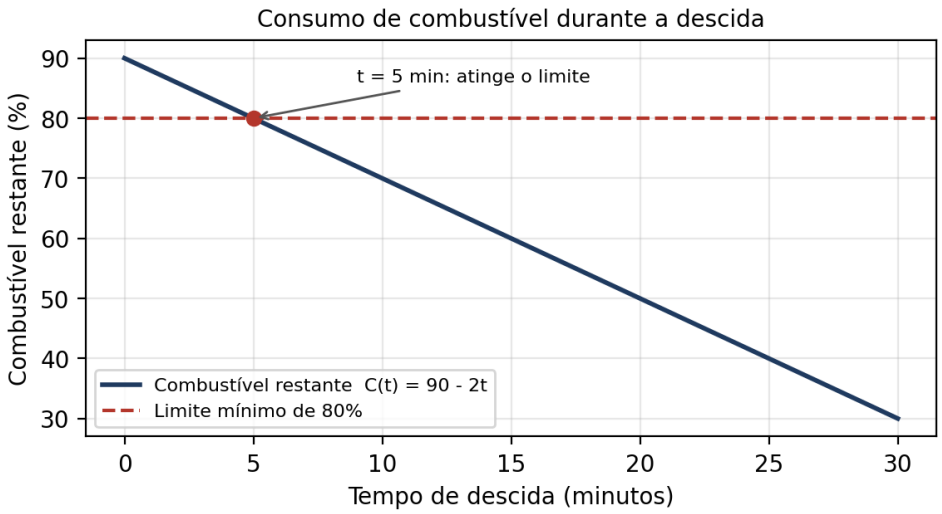


Figura 2 – Consumo de combustível durante a descida, com o limite mínimo de 80%. Fonte: elaborada pela equipe (2026).

Análise do gráfico. A reta tem inclinação negativa: a cada minuto o módulo perde 2 pontos percentuais. Ela parte de 90% (t = 0), cruza o limite de 80% em t = (90 − 80) / 2 = **5 minutos** e chega a 30% em t = 30. Se a taxa *a* aumenta (frenagem mais intensa), a reta fica mais inclinada e o limite é atingido mais cedo (com a = 3, em cerca de 3,3 min; com a = 1, em 10 min). Se C_0 aumenta, a reta sobe e a janela de operação segura fica maior.

Relação com as decisões do MGPEB. O mesmo limite de 80% aparece na regra lógica (`combustivel_ok`) e na função. A regra decide com o nível atual; a função projeta *quanto tempo* resta até esse nível ser violado. Com isso o sistema pode definir o melhor momento para acionar os retrofoguetes ou limitar quantos módulos pousam ao mesmo tempo: se a janela de 5 minutos não comporta mais um pouso, o módulo seguinte é adiado. É um modelo simplificado, pois o consumo real de um foguete depende da massa, da velocidade e da atmosfera e não é linear. Para o escopo da atividade, a função linear permite análise clara e previsão direta.

```
def calcular_combustivel(combustivel_inicial, taxa_consumo, tempo):
    return combustivel_inicial - taxa_consumo * tempo      # C(t) = C0 - a*t

tempo = np.linspace(0, 30, 31)                  # vetor: 0, 1, 2 ... 30 minutos
combustivel = calcular_combustivel(90, 2, tempo)
tempo_limite = (90 - 80) / 2                    # 5 minutos
```

O NumPy e o Matplotlib são usados **somente** nesta etapa (vetor de tempo e gráfico). Toda a lógica de decisão, as estruturas de dados e os algoritmos usam apenas Python básico, conforme exige a atividade.

8. Contexto histórico e arquitetura de computadores

O MGPEB só é possível porque a computação evoluiu em etapas. Os primeiros computadores de propósito geral, como o ENIAC (1945), eram máquinas enormes, feitas com válvulas, e consumiam muita energia. Os transistores e, depois, os circuitos integrados permitiram miniaturizar e tornar as máquinas confiáveis o bastante para voar. O computador de orientação da Apollo foi um dos primeiros a usar circuitos integrados e tinha cerca de 2 mil palavras de memória apagável e 36 mil de memória fixa. Com tão pouco, era essencial um software que priorizasse tarefas, exatamente a ideia que a nossa fila de pouso por prioridade reproduz em escala didática.

De lá para hoje, os mainframes deram lugar aos microprocessadores e aos sistemas embarcados dedicados. As sondas atuais em Marte usam processadores endurecidos contra radiação, como o RAD750 (derivado do PowerPC 750), com desempenho muito inferior ao de um celular, porque nesse ambiente **confiabilidade vale mais que velocidade**.

Limitações típicas de uma missão em Marte e como influenciaram nossas escolhas:

- **Memória limitada:** usamos listas simples e uma cópia auxiliar pequena, sem estruturas pesadas.
- **Processamento restrito:** algoritmos simples, previsíveis e fáceis de verificar (seleção e inserção) em vez de soluções mais sofisticadas que só compensam com grandes volumes de dados.
- **Consumo de energia:** poucas operações por ciclo de decisão e nenhuma estrutura recalculada sem necessidade (a lista alfabética é ordenada uma única vez).
- **Radiação e falhas:** regra de decisão conservadora, com AND de quatro condições: na dúvida, o pouso é adiado e o evento fica registrado.

9. Governança ambiental, social e corporativa (ESG)

Os princípios ESG, desenvolvidos para empresas na Terra, oferecem um bom arcabouço para pensar uma colônia em Marte com responsabilidade, seguindo a lógica do *triple bottom line* (pessoas, planeta e lucro/prosperidade), de Elkington.

Ambiental: escolha da área de pouso. Os critérios devem incluir terreno estável e plano, boa incidência solar, proximidade de gelo de água para uso local e **distância de regiões de interesse científico ou de possível presença de água líquida**, para evitar contaminação biológica e preservar o ambiente marciano. No código, isso aparece na variável `area_disponivel`: sem área adequada, não há pouso.

Ambiental: energia, resíduos e recursos locais. A energia deve vir principalmente de painéis solares com armazenamento, mantendo o módulo de Energia como prioridade máxima. A água e os minerais são escassos, então devem ser usados em ciclo fechado: reaproveitamento de água, reciclagem de resíduos e extração local limitada ao necessário.

Social: a vida da tripulação vem antes de qualquer outra meta. Por isso Suporte à Vida e Energia têm a maior prioridade e criticidade. O acesso a recursos como energia, água e atendimento médico deve ser equitativo, com regras claras de convivência e de trabalho.

Governança: decisões sobre tecnologia e recursos precisam de transparência e rastreabilidade. A **pilha de eventos** do protótipo funciona como uma trilha de auditoria: cada decisão de pouso (autorizada ou não) fica registrada, do mais recente ao mais antigo. Propomos ainda revisão humana obrigatória sempre que houver alerta, um conselho da colônia com representação dos grupos de trabalho e prestação de contas periódica à Terra.

10. Conclusão

O MGPEB integra, em um único protótipo, portas lógicas, programação em Python, estruturas lineares, algoritmos de busca e ordenação, modelagem de funções, história da computação e ESG. O sistema autorizou o pouso de Energia e Habitação e colocou Suporte à Vida, Logística e Científico em alerta, cada um por um motivo identificado e registrado. Como melhorias futuras, a fila poderia usar `collections.deque` (a remoção com `pop(0)` em lista é O(n)), os módulos poderiam ser dicionários ou classes (em vez de índices numéricos, o que reduziria erros) e a função de consumo poderia incluir a massa de cada módulo.

Anexo A – Estruturas de dados no projeto, com exemplos concretos

A.1 Lista de listas (modulos). Cada módulo é uma lista com nove posições e todos ficam dentro de uma lista maior:

Índice	0	1	2	3	4	5	6	7	8
Campo	Nome	Prior.	Comb. (%)	Massa (t)	Critic.	Chegada	Atmosf. OK	Área livre	Sensores OK
Exemplo	Habitação	4	85	20	4	10:00	True	True	True

```
modulos[1][0]   # "Energia"  (nome do segundo módulo)
modulos[3][2]   # 78         (combustível do Científico)
```

A.2 Fila (FIFO) - fila_pouso. Os módulos são enfileirados com `append()`, reordenados por prioridade e depois atendidos com `pop(0)`, sempre retirando o primeiro. O tamanho da fila vai de 5 a 0 durante o processamento:

Momento	Conteúdo da fila (início → fim)
Antes de ordenar	Habitação, Energia, Suporte à Vida, Científico, Logística
Depois do Selection Sort	Energia, Suporte à Vida, Habitação, Logística, Científico
Após 1º pop(0) (sai Energia)	Suporte à Vida, Habitação, Logística, Científico
Após 2º pop(0) (sai Suporte à Vida)	Habitação, Logística, Científico
Ao final do processamento	(vazia)

A.3 Pilha (LIFO) - pilha_eventos. Cada decisão é empilhada com `append()`. Para exibir do mais recente ao mais antigo, a lista é lida de trás para frente com `reversed()`. Conteúdo final após a simulação:

Posição	Evento
TOPO (mais recente)	Científico: pouso não autorizado
	Logística: pouso não autorizado
	Habitação: pouso autorizado
	Suporte à Vida: pouso não autorizado
BASE (mais antigo)	Energia: pouso autorizado

A.4 Listas auxiliares. `modulos_pousados` termina com [Energia, Habitação]; `modulos_alerta` com [Suporte à Vida, Logística, Científico]; `modulos_alfabeticos` é uma cópia de `modulos` ordenada por nome, usada na busca binária.

Anexo B – Dicionário de variáveis do código

Variável	Tipo	O que representa	Onde é usada
modulos	list de list	Cadastro dos 5 módulos (9 campos cada)	Dados e busca linear
fila_pouso	list (fila)	Módulos aguardando autorização de pouso	selection_sort, processar_pousos
modulos_pousados / modulos_alerta	list	Resultado das autorizações e dos alertas	processar_pousos, exibir_resumo
pilha_eventos	list (pilha)	Histórico de decisões	processar_pousos, exibir_pilha
modulos_alfabeticos	list	Cópia ordenada por nome	insertion_sort_nome, busca_binaria
modulo	list	Módulo atual em um laço ou fila	Vários laços e funções
combustivel_ok, atmosfera_ok, area_disponivel, sensores_ok	bool	Condições de segurança do pouso	Regra AND/NOT
total, contador	int	Total de módulos e número do módulo atual	Mensagens [1/5], [2/5]...
n, i, j, maior	int	Tamanho da lista, índices dos laços, posição do maior valor	selection_sort
atual	list	Elemento que está sendo inserido	insertion_sort_nome
inicio, fim, meio	int	Limites e ponto médio da busca	busca_binaria
nome_busca, nome_meio	str	Nome procurado e nome do meio (em minúsculas via casefold)	busca_binaria
combustivel_inicial, taxa_consumo, limite_combustivel	int	$C_0 = 90$, a = 2, limite = 80	exibir_grafico_combustivel
tempo, combustivel	array NumPy	Vetor de tempo (0 a 30) e combustível calculado	exibir_grafico_combustivel
tempo_limite	float	Minutos até atingir o limite (5,0)	exibir_grafico_combustivel

Referências

ELKINGTON, John. *The Triple Bottom Line: Does It All Add Up?* Pearson, 2018.

FLOYD, Thomas. *Sistemas Digitais – Fundamentos e Aplicações*. 11. ed. São Paulo: Pearson, 2016.

GOODRICH, Michael; TAMASSIA, Roberto; GOLDWASSER, Michael. *Data Structures & Algorithms in Python*. Pearson, 2016.

GUTTAG, John. *Introduction to Computation and Programming Using Python*. Pearson, 2021.

LAFORGE, Linda; WAITE, Sue. *Mathematical Modeling and Applied Mathematics*. Pearson, 2018.

STALLINGS, William. *Arquitetura e Organização de Computadores*. 10. ed. São Paulo: Pearson, 2021.

TANENBAUM, Andrew; AUSTIN, Todd. *Estrutura e Organização de Computadores*. 6. ed. São Paulo: Pearson, 2020.